



Metaheuristic Optimization



**Heuristic Algorithms in Transport
and Finance**

The VRP and the CWS Heuristic

Prof. Dr. Angel A. Juan

Full Professor, Applied Stats and Operations Research Dept.

ajuanp@upv.es

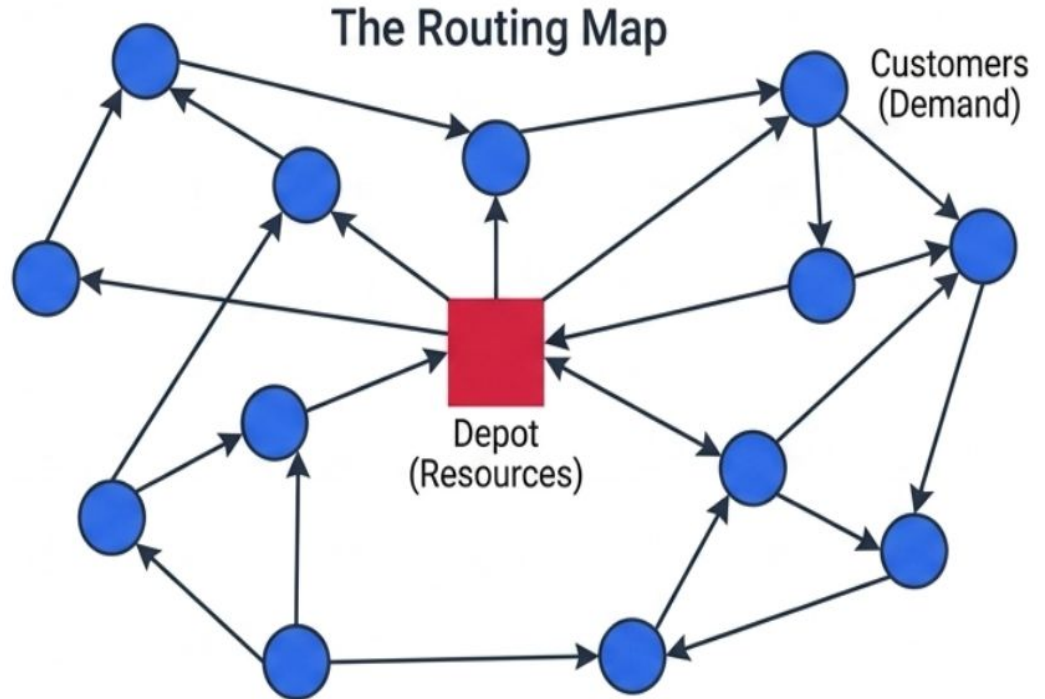
Routing resources to distributed demands is an NP-hard optimization challenge

The Objective

Establish a set of routes satisfying all constraints with minimum total travel cost, represented by $c(i, j)$.

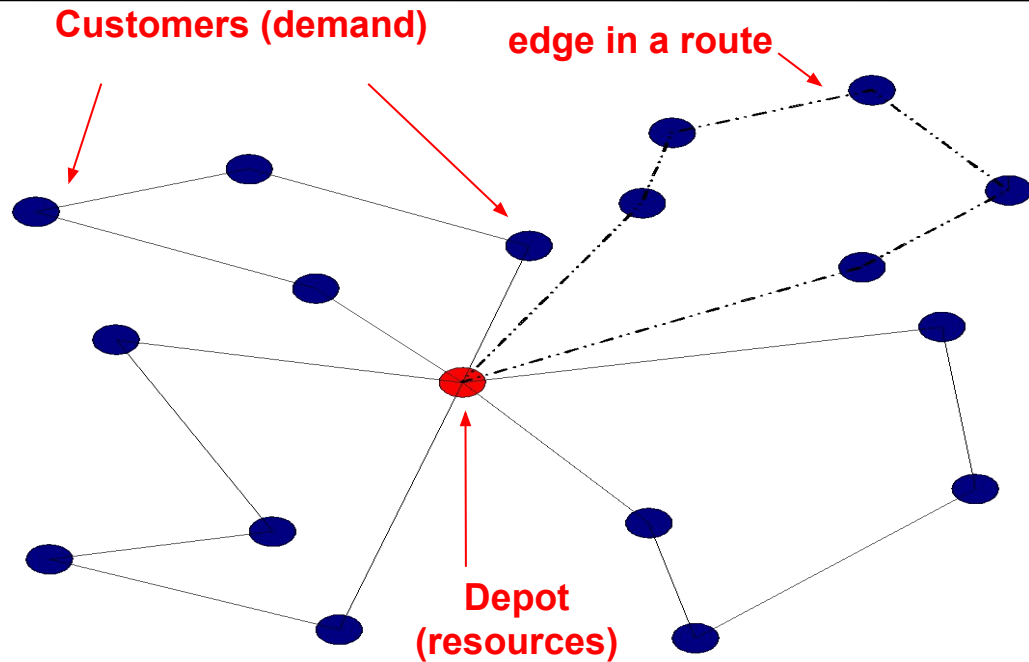
The Constraints

Vehicle capacity, maximum distance, time limits, and time windows.



The Vehicle Routing Problem (1/2)

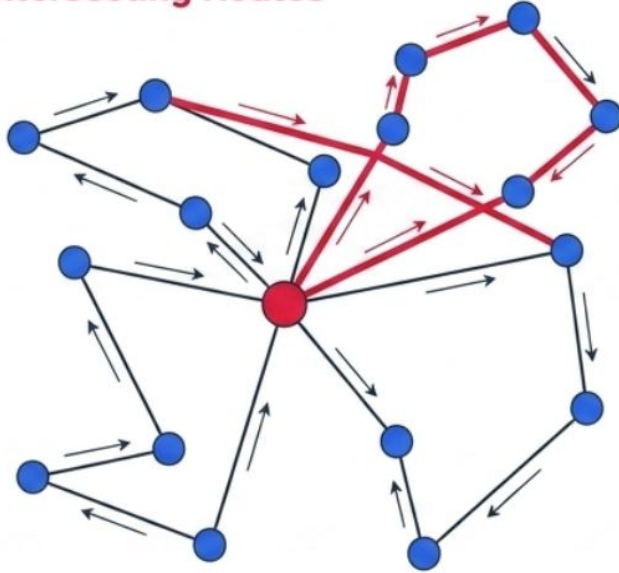
- The **Vehicle Routing Problem** (VRP) is a well-known NP-hard problem:
 - Depot, customers' demands, fleet of vehicles, etc.
 - Travel costs $c(i, j)$
 - Constraints: vehicle capacity, max. distance / time, time windows, etc.
- Goal: set of routes satisfying all constraints with **minimum cost**.
- Different **approaches** for the VRP (exact vs. approximate).



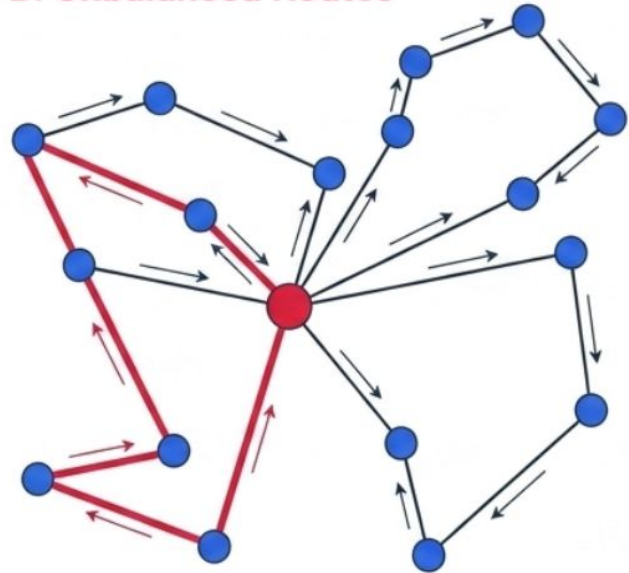
Real-world constraints prevent perfect mathematical models in advance

Diagnostic Route Matrix

Panel A: Intersecting Routes



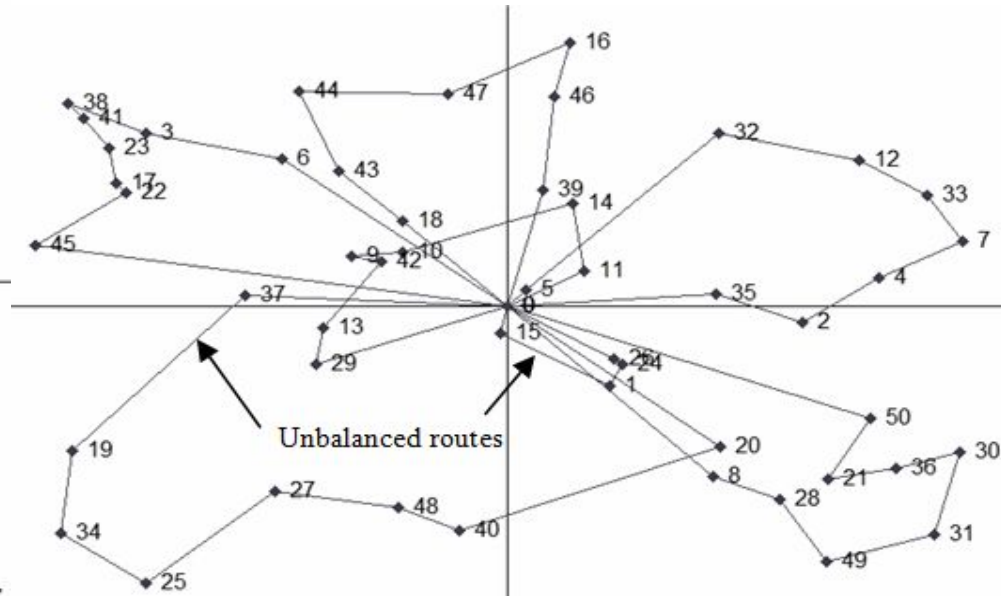
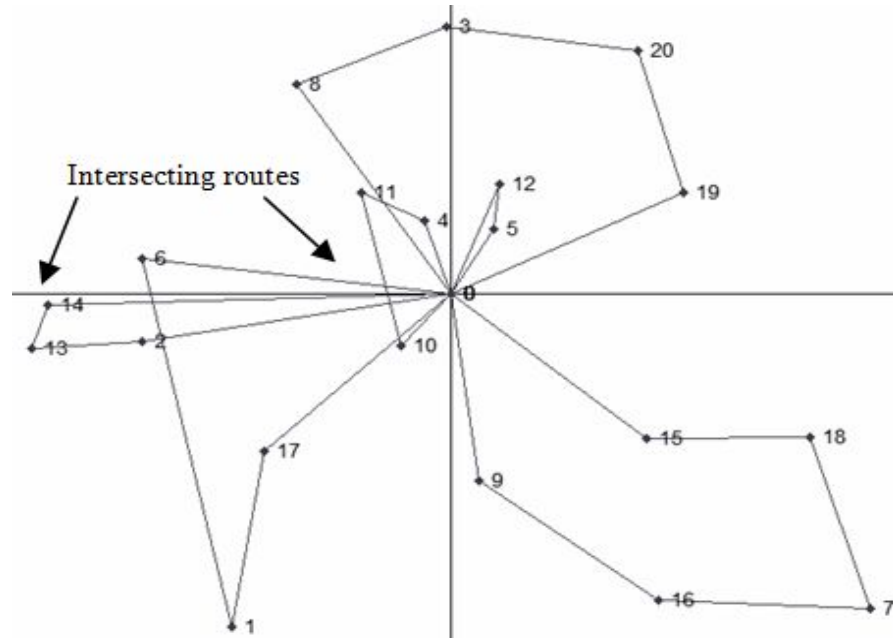
Panel B: Unbalanced Routes



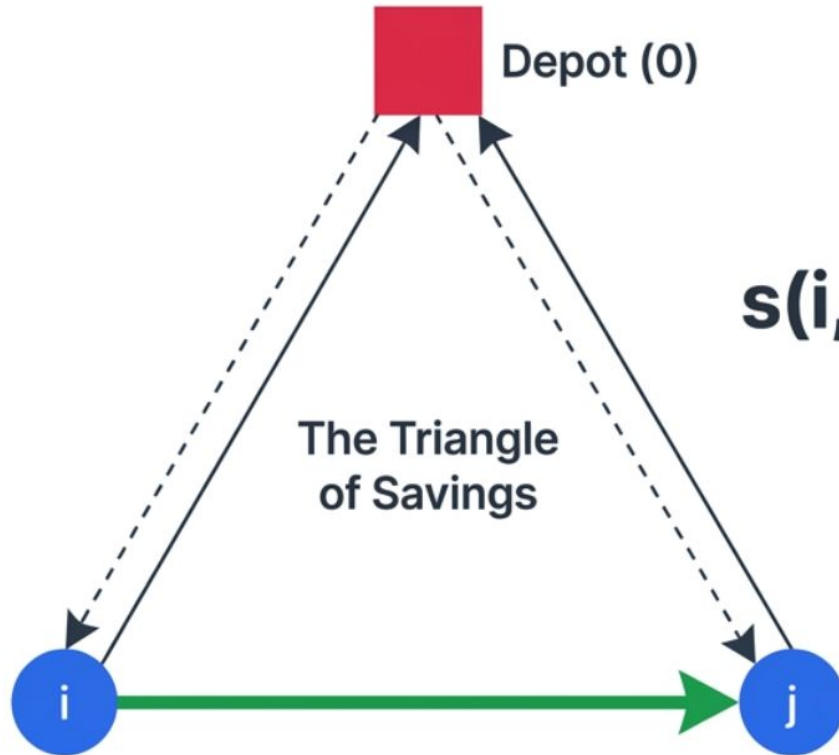
Because real-life scenarios cannot model all costs in advance, algorithms must rapidly generate many 'good' alternative solutions evaluated by a utility function rather than searching for one impossible exact perfect solution.

The Vehicle Routing Problem (2/2)

- Yes, but...: In real-life scenarios is not always possible to model all costs and constraints in advance (Kant et al 2008)
- Is it possible to generate many 'good' alternative solutions? (utility function)



Bypassing the depot generates quantifiable distance savings



$$s(i, j) = c(0, i) + c(0, j) - c(i, j)$$

The formula calculates exactly how much distance is saved by traveling directly between two clients rather than returning to base.

The Clarke & Wright Savings Heuristic provides high-quality approximate solutions

ORIGIN

Introduced by Clarke & Wright in 1964.

THE PARADIGM

Instead of calculating the perfect overall route, calculate the relative savings of connecting any two specific nodes.

THE RESULT

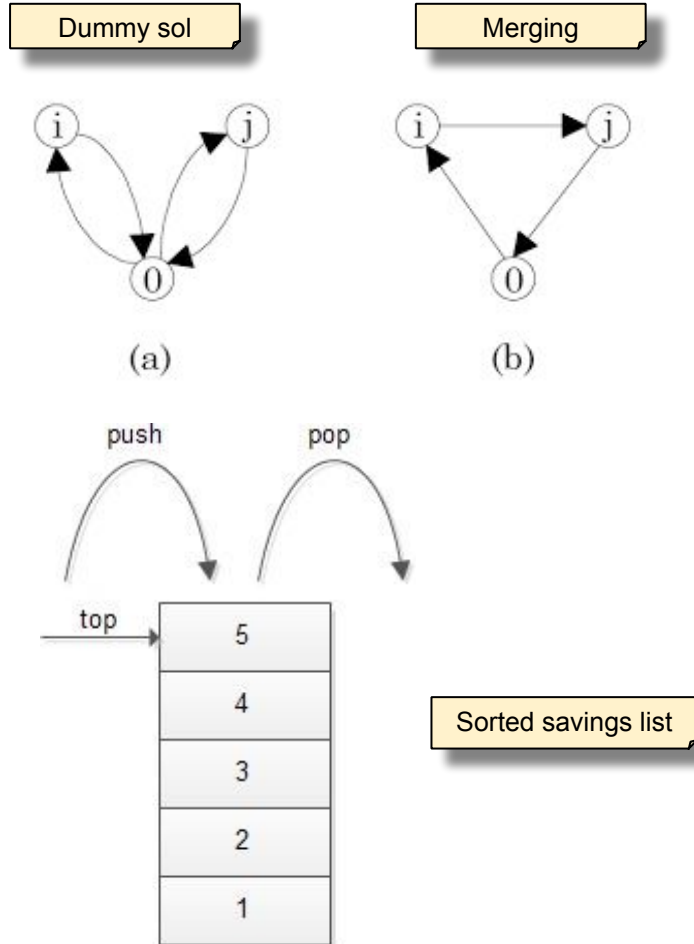
Provides highly efficient solutions in fractions of a second.

5%

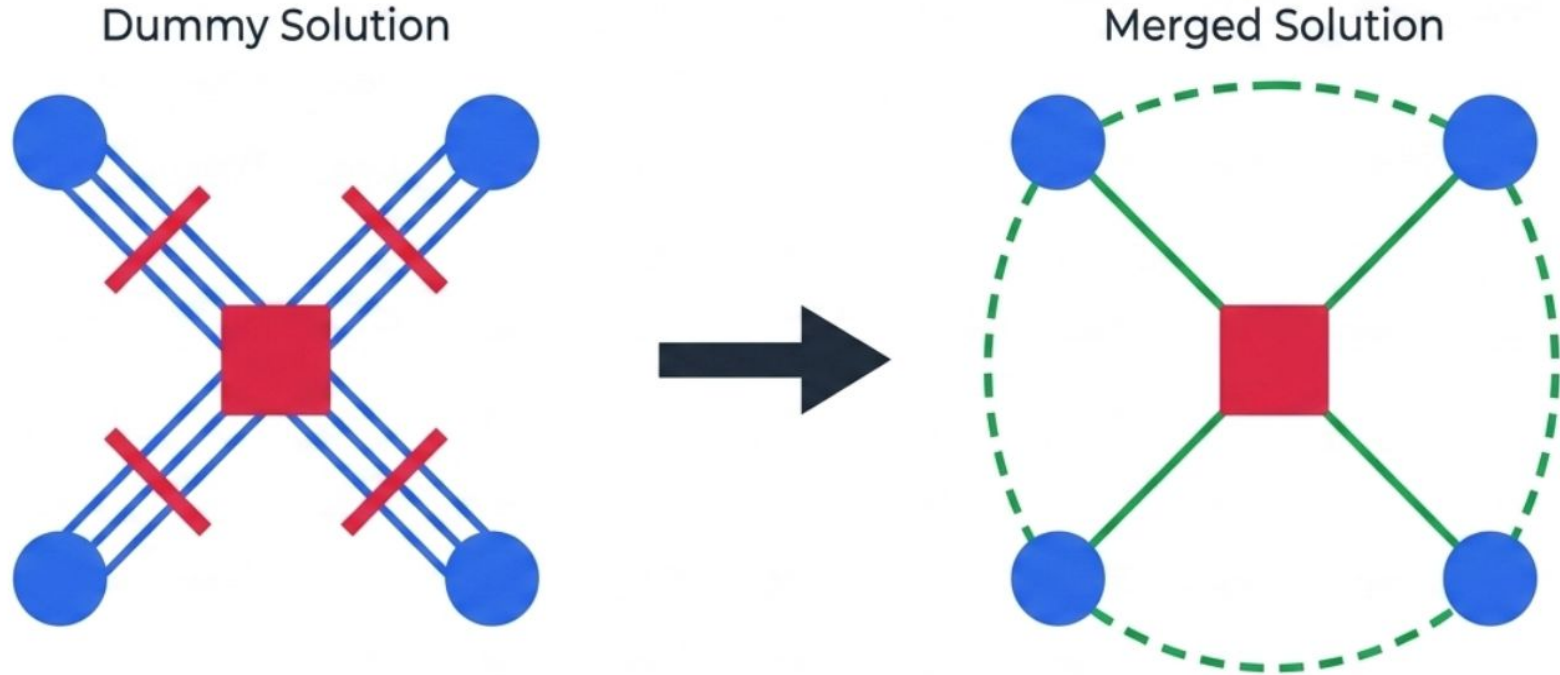
Consistently yields an average gap of under 5% from the best-known theoretical solutions.

The Savings Heuristic for the VRP (1/3)

- **Savings (CWS) heuristic** (Clarke & Wright 1964).
- CWS heuristic:
 1. For each pair of nodes i and j , calculate the **savings**, $s(i, j)$, associated with the edge connecting them, where:
$$s(i, j) = c(0, i) + c(0, j) - c(i, j)$$
 2. Construct a **list of edges**, sorting the edges according to their associated savings.
 3. Construct an **initial feasible solution** by routing a vehicle to each client node.
 4. Select the first edge in the savings list and, if no constraint is violated, **merge the routes** that it connects.
 5. Repeat step 4 until the savings list is empty.
- Provides '**good**' solutions (average gap between 5% and 10%) in **milliseconds**!

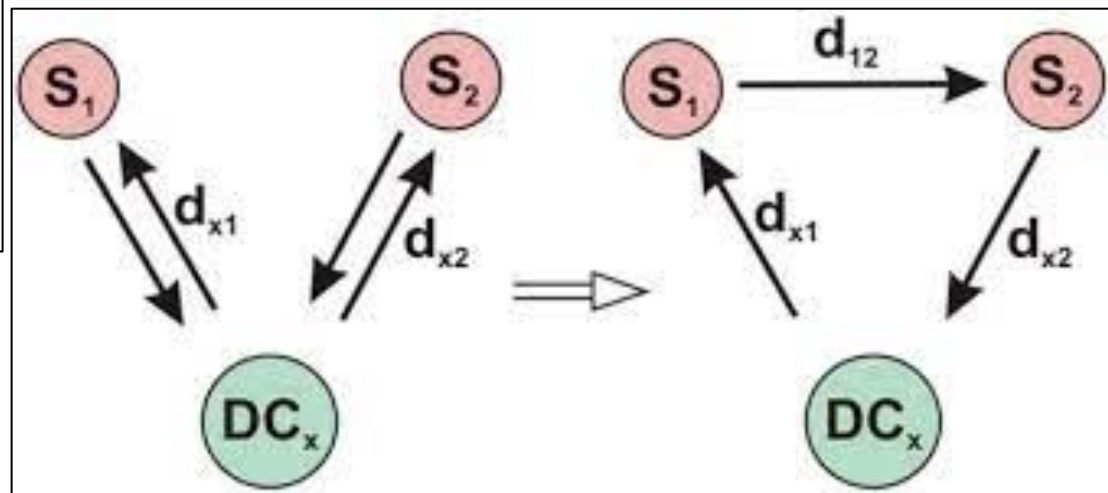
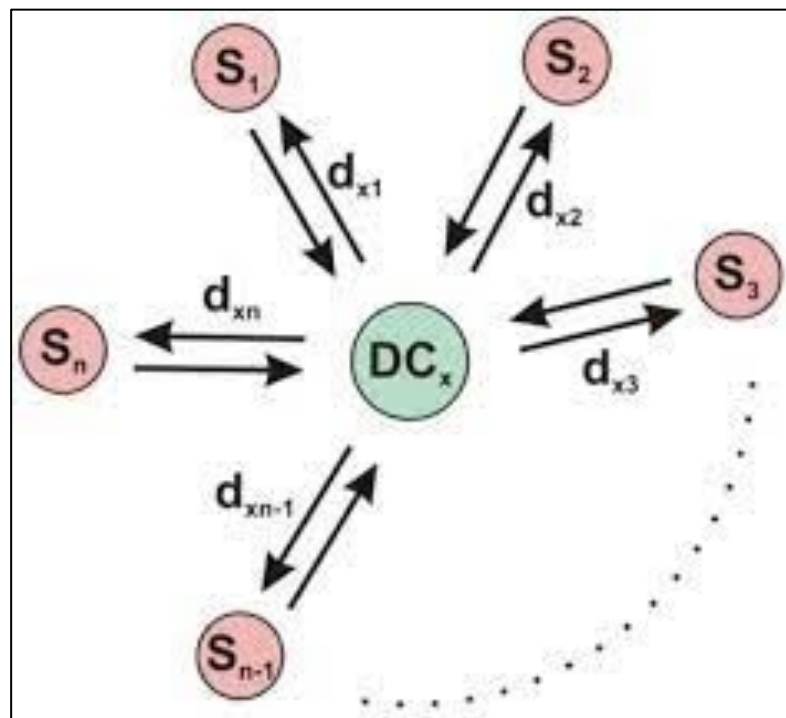


Dummy solutions merge into optimized networks by eliminating redundant depot returns

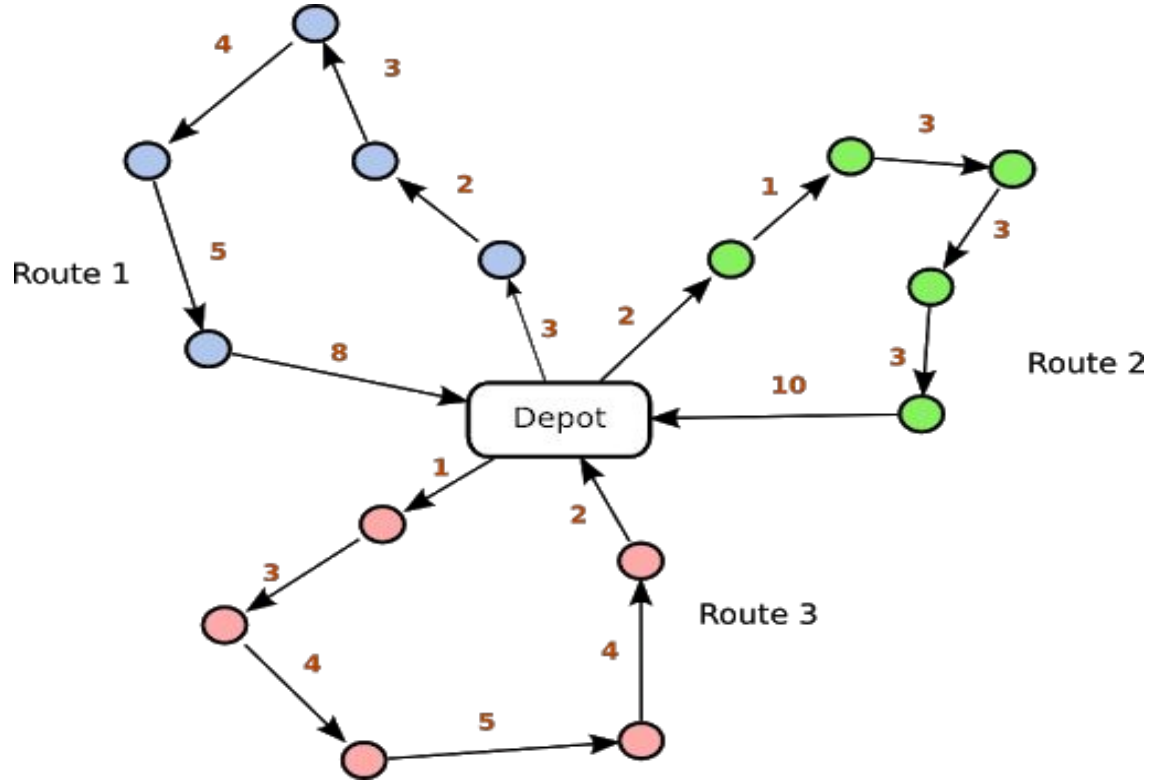
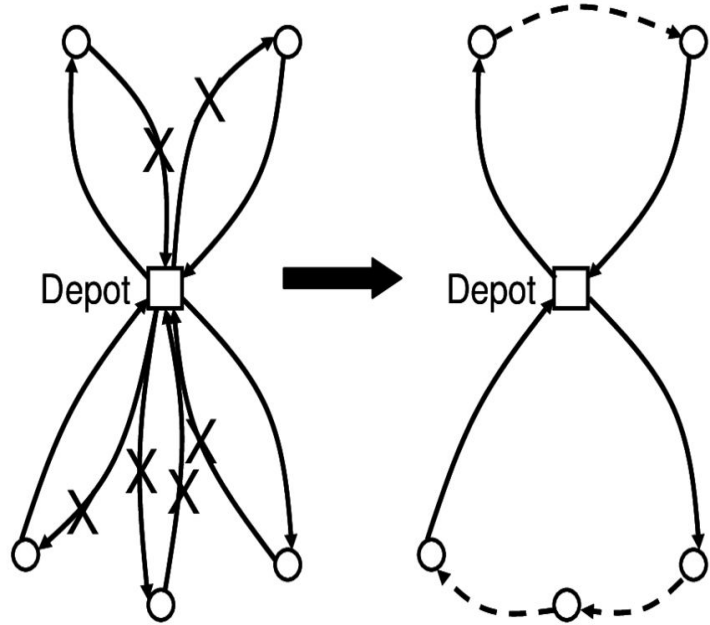


The algorithm systematically prunes the most wasteful return trips first.

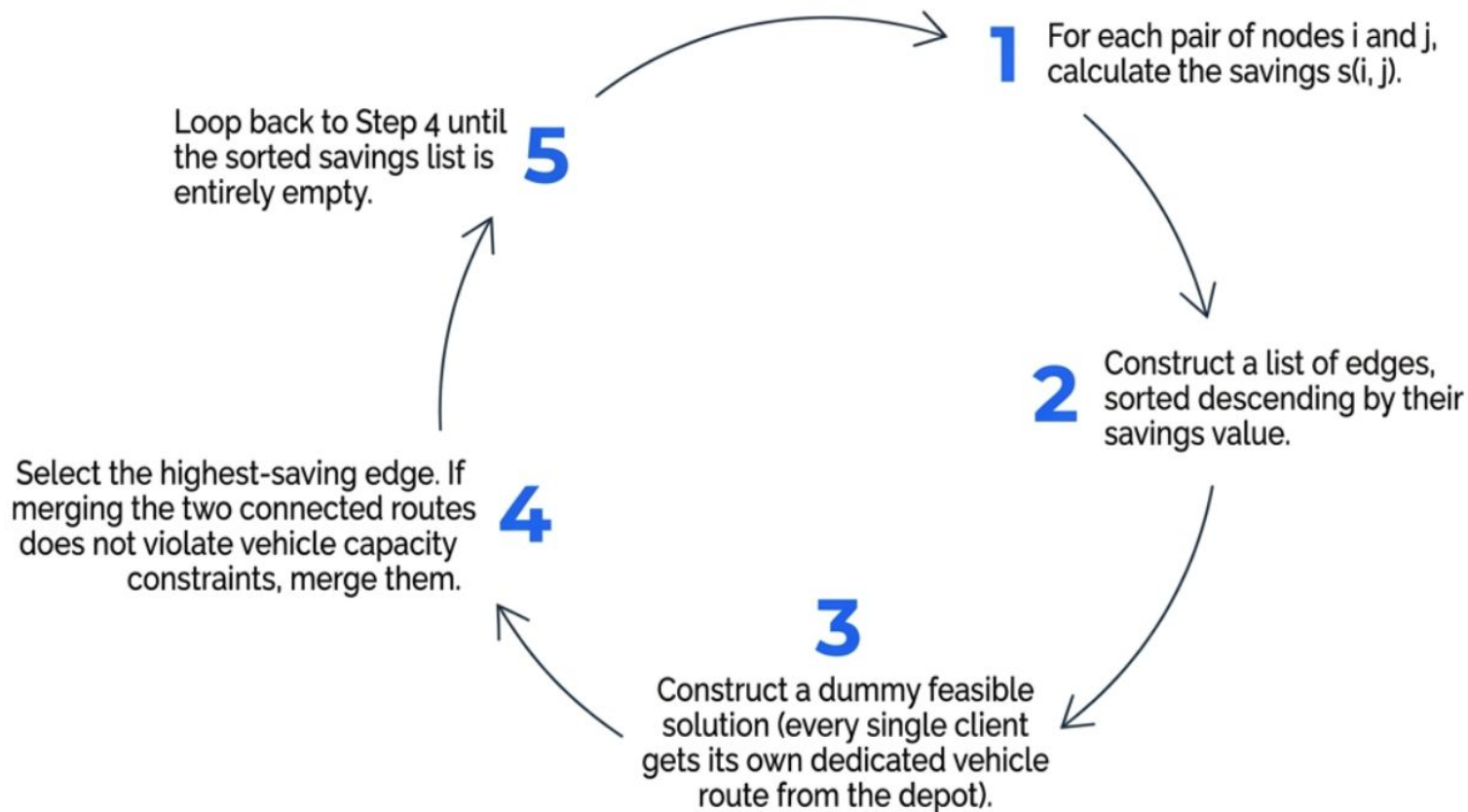
The Savings Heuristic for the VRP (2/3)



The Savings Heuristic for the VRP (3/2)



The five-step heuristic evaluates **savings** and merges routes iteratively



vrp_objects.py Classes Node and Edge

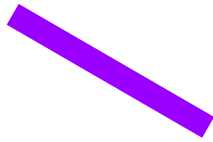


```
1  class Node:
2
3      def __init__(self, ID, x, y, demand):
4          self.ID = ID # node identifier (depot ID = 0)
5          self.x = x # Euclidean x-coordinate
6          self.y = y # Euclidean y-coordinate
7          self.demand = demand # demand (is 0 for depot and positive for others)
8          self.inRoute = None # route to which node belongs
9          self.isInterior = False # an interior node is not connected to depot
10         self.dnEdge = None # edge (arc) from depot to this node
11         self.ndEdge = None # edge (arc) from this node to depot
12
13
14  class Edge:
15
16      def __init__(self, origin, end):
17          self.origin = origin # origin node of the edge (arc)
18          self.end = end # end node of the edge (arc)
19          self.cost = 0.0 # edge cost
20          self.savings = 0.0 # edge savings (Clarke & Wright)
21          self.invEdge = None # inverse edge (arc)
```

i

j

0

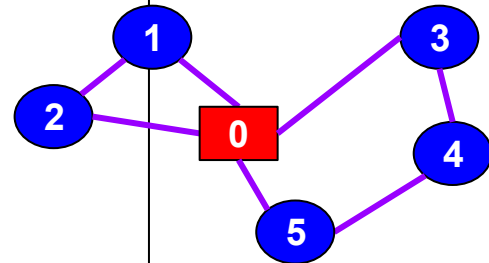
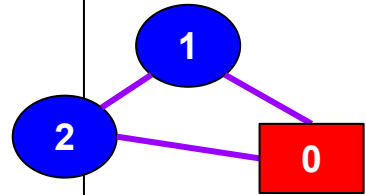


```
24 class Route:
```

```
25
26 def __init__(self):
27     self.cost = 0.0 # cost of this route
28     self.edges = [] # sorted edges in this route
29     self.demand = 0.0 # total demand covered by this route
30
31 def reverse(self): # e.g. 0 -> 2 -> 6 -> 0 becomes 0 -> 6 -> 2 -> 0
32     size = len(self.edges)
33     for i in range(size):
34         edge = self.edges[i]
35         invEdge = edge.invEdge
36         self.edges.remove(edge)
37         self.edges.insert(0, invEdge)
```

```
38
39
40 class Solution:
```

```
41
42 last_ID = -1 # counts the number of solutions, starts with 0
43
44 def __init__(self):
45     Solution.last_ID += 1
46     self.ID = Solution.last_ID
47     self.routes = [] # routes in this solution
48     self.cost = 0.0 # cost of this solution
49     self.demand = 0.0 # total demand covered by this solution
```



cws_heuristic.py Reading an Instance Data



```
1  """ Clarke & Wright Savings Algorithm, 2023 (c) Angel A. Juan """
2
3  from vrp_objects import Node, Edge, Route, Solution
4  from vrp_visualization import plot_map
5  import math
6  import operator
7
8  """ DICTIONARY OF INSTANCE NAMES AND ASSOCIATED VEHICLE CAPACITIES """
9  instances = {'A-n32-k5': 100, 'A-n38-k5': 100, 'A-n45-k7': 100, 'A-n55-k9': 100, 'A-n60-k9': 100,
10             'A-n61-k9': 100, 'A-n65-k9': 100, 'A-n80-k10': 100, 'B-n50-k7': 100, 'B-n52-k7': 100, 'B-n57-k9': 100,
11             'B-n78-k10': 100, 'E-n22-k4': 6000, 'E-n30-k3': 4500, 'E-n33-k4': 8000, 'E-n51-k5': 160, 'E-n76-k7': 220,
12             'E-n76-k10': 140, 'E-n76-k14': 100, 'F-n45-k4': 2010, 'F-n72-k4': 30000, 'F-n135-k7': 2210,
13             'M-n101-k10': 200, 'M-n121-k7': 200, 'P-n22-k8': 3000, 'P-n40-k5': 140, 'P-n50-k10': 100,
14             'P-n55-k15': 70, 'P-n65-k10': 130, 'P-n70-k10': 135, 'P-n76-k4': 350, 'P-n76-k5': 280, 'P-n101-k4': 400}
15
16  """ USER SELECTS THE INSTANCE """
17  instanceName = 'A-n80-k10'
18  if instanceName in instances:
19      vehCap = instances[instanceName]
20  else:
21      print(f"Instance '{instanceName}' not found in the dictionary.")
22
23  """ READ NODES COORDINATES AND DEMANDS """
24  fileName = f'vrp-data/{instanceName}_input_nodes.txt'
25  with open(fileName) as instance:
26      nodes = []
27      for index, line in enumerate(instance):
28          x, y, demand = tuple(float(value) for value in line.split())
29          aNode = Node(index, x, y, demand)
30          nodes.append(aNode)
31
32  """ PLOT MAP OF NODES WITHOUT ROUTES """
33  plot_map(nodes, instanceName=instanceName, show_routes=False)
```

Make sure you use the right **vehCap** when you test a different instance.

Data on instances is available at:
https://drive.google.com/drive/folders/1et0VafwaOEEABDbzhlia9pU7vCW4IL_6?usp=drive_link

JORS Table with Instances and CWS Solutions

Table 1 Results for 33 standard benchmarks (multi-thread C implementation)

Instance	# nodes	vCap	CWS Sol.	Gap BSK-CWS (%)	Best-known Sol.	BKS (int)	# routes BKS	Source	Gap BKS-OBS (%)	Our Best Sol.	# routes OBS	Time (s)
A-n32-k5	32	100	843.69	7.09	787.81	784	5	2	-0.09	787.08	5	1
A-n38-k5	38	100	768.14	4.63	734.18	730	5	2	-0.03	733.95	5	1
A-n45-k7	45	100	1 199.98	4.59	1 175.22	1 170	7	2	-0.04	1 146.77	7	1
A-n55-k9	55	100	1 099.84	2.36	1 075.88	1 070	9	2	0.00	1 074.46	9	1
A-n60-k9	60	100	1 421.88	4.87	1 378.22	1 370	9	2	0.00	1 355.80	9	1
A-n61-k9	61	100	1 102.23	6.08	1 055.88	1 050	9	2	0.00	1 039.08	9	2
A-n65-k9	65	100	1 239.42	4.89	1 195.88	1 190	9	2	0.00	1 181.69	9	2
A-n80-k10	80	100	1 860.94	5.35	1 744.78	1 740	10	1	0.00	1 766.50	10	178
B-n50-k7	50	100	748.80	0.54	744.78	741	7	1	-0.07	744.23	7	1
B-n52-k7	52	100	764.90	1.98	750.08	747	7	1	-0.02	749.96	7	1
B-n57-k9	57	100	1 653.42	3.10	1 603.63	1 598	9	1	-0.08	1 602.28	9	1
B-n78-k10	78	100	1 264.56	2.87	1 229.27	1 221	10	1	-0.11	1 227.90	10	9
E-n22-k4	22	6 000	388.77	3.59	375.28	375	4	2	0.00	375.28	4	1
E-n30-k3	30	4 500	534.45	-0.25	535.80	534	3	2	-5.75	505.01	4	1
E-n33-k4	33	8 000	843.10	0.52	838.72	835	4	2	-0.13	837.67	4	1
E-n51-k5	51	160	584.64	11.37	524.94	521	5	1	-0.06	524.61	5	1
E-n76-k7	76	220	737.74	7.29	687.60	682	7	2	0.00	687.60	7	7
E-n76-k10	76	140	900.26	7.51	837.36	830	10	1	-0.25	835.28	10	231
E-n76-k14	76	100	1 073.43	4.55	1 026.71	1 021	14	1	-0.22	1 024.40	15	32
F-n45-k4	45	2 010	739.02	1.99	724.57	721	4	3	-0.14	723.54	4	5
F-n72-k4	72	30 000	256.19	2.97	248.81	237	4	3	-2.75	241.97	4	2
F-n135-k7	135	2 210	1 219.32	4.16	1 170.65	1 159	7	1	-0.51	1 164.73	7	131
M-n101-k10	101	200	833.51	1.67	819.81	820	10	3	-0.03	819.56	10	0
M-n121-k7	121	200	1 068.14	2.20	1 045.16	1 034	7	1	-0.12	1 043.88	7	107
P-n22-k8	22	3 000	590.62	-1.80	601.42	603	8	3	-2.10	588.79	9	1
P-n40-k5	40	140	518.37	12.27	461.73	458	5	3	0.00	461.73	5	1
P-n50-k10	50	100	734.32	4.97	699.56	696	10	3	0.00	699.56	10	2
P-n55-k15	55	70	978.07	-1.35	991.48	98	15	3	0.00	978.07	15	1
P-n65-k10	65	130	851.67	6.90	796.67	79	10	3	0.00	796.67	10	1
P-n70-k10	70	135	896.86	8.05	830.02	82	10	3	0.00	830.02	10	1
P-n76-k4	76	350	689.13	15.20	598.22	59	4	3	0.00	598.22	4	1
P-n76-k5	76	280	698.51	9.99	635.04	62	5	3	0.00	635.04	5	1
P-n101-k4	101	400	765.38	10.56	692.28	68	4	3	0.00	692.28	4	1
Averages	63			4.87					-0.34			29

The A-n80-k10 uses a vehCap = 100, and its CWS cost is 1860.94.

Juan, A. A., Faulín, J., Jorba, J., Riera, D., Masip, D., & Barrios, B. (2011). On the use of Monte Carlo simulation, cache and splitting techniques to improve the Clarke and Wright savings heuristics. *Journal of the Operational Research Society*, 62(6), 1085-1097.

cws_heuristic.py Creating the Edges and List

```
34 """ CREATE EDGES CONNECTING EACH CUSTOMER WITH THE DEPOT (NODE 0) """
35 depot = nodes[0] # node 0 is the depot
36 for node in nodes[1:]: # excludes de depot
37     dnEdge = Edge(depot, node) # creates the (depot, node) edge (arc)
38     ndEdge = Edge(node, depot)
39     dnEdge.invEdge = ndEdge # sets the inverse edge (arc)
40     ndEdge.invEdge = dnEdge
41     # compute the Euclidean distance as cost
42     dnEdge.cost = math.sqrt((node.x - depot.x) ** 2 + (node.y - depot.y) ** 2)
43     ndEdge.cost = dnEdge.cost # assume symmetric costs
44     # save in node a reference to the (depot, node) edge (arc)
45     node.dnEdge = dnEdge
46     node.ndEdge = ndEdge
47
48
49 """ CREATE EDGES CONNECTING PAIRS OF CUSTOMERS AND COMPUTE SAVINGS LIST """
50 savingsList = []
51 for i in range(1, len(nodes) - 1): # excludes de depot
52     iNode = nodes[i]
53     for j in range(i + 1, len(nodes)):
54         jNode = nodes[j]
55         ijEdge = Edge(iNode, jNode) # creates the (i, j) edge
56         jiEdge = Edge(jNode, iNode)
57         ijEdge.invEdge = jiEdge # sets the inverse edge (arc)
58         jiEdge.invEdge = ijEdge
59         # compute the Euclidean distance as cost
60         ijEdge.cost = math.sqrt((jNode.x - iNode.x) ** 2 + (jNode.y - iNode.y) ** 2)
61         jiEdge.cost = ijEdge.cost # assume symmetric costs
62         # compute savings as proposed by Clark & Wright
63         ijEdge.savings = iNode.ndEdge.cost + jNode.dnEdge.cost - ijEdge.cost
64         jiEdge.savings = ijEdge.savings
65         # save one edge in the savings list
66         savingsList.append(ijEdge)
67
68 # sort the list of edges from higher to lower savings
69 savingsList.sort(key=operator.attrgetter("savings"), reverse=True)
```

```

69 """ CONSTRUCT THE DUMMY SOLUTION """
70 sol = Solution()
71 for node in nodes[1:]: # excludes the depot
72     dnEdge = node.dnEdge # get the (depot, node) edge
73     ndEdge = node.ndEdge
74     dndRoute = Route() # construct the route (depot, node, depot)
75     dndRoute.edges.append(dnEdge)
76     dndRoute.demand += node.demand
77     dndRoute.cost += dnEdge.cost
78     dndRoute.edges.append(ndEdge)
79     dndRoute.cost += ndEdge.cost
80     node.inRoute = dndRoute # save in node a reference to its current route
81     node.isInterior = False # this node is currently exterior (connected to depot)
82     sol.routes.append(dndRoute) # add this route to the sol
83     sol.cost += dndRoute.cost
84     sol.demand += dndRoute.demand
85
86
87 """ PLOT MAP OF NODES WITH ROUTES """
88 plot_map(nodes, sol=sol, instanceName=instanceName)
89
90 """ FUNCTION TO CHECK IF MERGING CONDITIONS ARE MET """
91 def checkMergingConditions(iNode, jNode, iRoute, jRoute):
92     # condition 1: iRoute and jRoute are not the same route object
93     if iRoute == jRoute:
94         return False
95     # condition 2: both nodes are exterior nodes in their respective routes
96     if iNode.isInterior or jNode.isInterior:
97         return False
98     # condition 3: demand after meging can be covered by a single vehicle
99     if vehCap < iRoute.demand + jRoute.demand:
100         return False
101     # else, merging is feasible
102     return True
103
104 """ FUNCTION THAT RETURNS THE EDGE IN A ROUTE CONTAINING AN EXTERNAL NODE AND THE DEPOT """
105 def getDepotEdge(aRoute, aNode):
106     origin = aRoute.edges[0].origin
107     end = aRoute.edges[0].end
108     if ((origin == aNode and end == depot) or (origin == depot and end == aNode)):
109         return aRoute.edges[0]
110     else: # return last edge in aRoute
111         return aRoute.edges[-1]
112

```

```
113 """ ITERATIVE ROUTE MERGING PROCESS """
114 while len(savingsList) > 0: # list is not empty
115     ijEdge = savingsList.pop(0) # select the next edge from the list
116     # determine the nodes i < j that define the edge
117     iNode = ijEdge.origin
118     jNode = ijEdge.end
119     # determine the routes associated with each node
120     iRoute = iNode.inRoute
121     jRoute = jNode.inRoute
122     # check if merge is possible
123     isMergeFeasible = checkMergingConditions(iNode, jNode, iRoute, jRoute)
124     # if all necessary conditions are satisfied, merge
125     if isMergeFeasible:
126         # iRoute will contain either edge (depot, i) or edge (i, depot)
127         iEdge = getDepotEdge(iRoute, iNode) # iEdge is either (0,i) or (i,0)
128         # remove iEdge from iRoute and update iRoute cost
129         iRoute.edges.remove(iEdge)
130         iRoute.cost -= iEdge.cost
131         # if there are multiple edges in iRoute, then i will be interior
132         if len(iRoute.edges) > 1:
133             iNode.isInterior = True
134         # if new iRoute does not start at 0 it must be reversed
135         if iRoute.edges[0].origin != depot:
136             iRoute.reverse()
137         # jRoute will contain either edge (depot, j) or edge (j, depot)
138         jEdge = getDepotEdge(jRoute, jNode) # jEdge is either (0,j) or (j,0)
139         # remove jEdge from jRoute and update jRoute cost
140         jRoute.edges.remove(jEdge)
141         jRoute.cost -= jEdge.cost
```

```
142     # if there are multiple edges in jRoute, the j will be interior
143     if len(jRoute.edges) > 1:
144         jNode.isInterior = True
145     # if new jRoute starts at 0 it must be reversed
146     if jRoute.edges[0].origin == depot:
147         jRoute.reverse()
148     # add ijEdge to iRoute
149     iRoute.edges.append(ijEdge)
150     iRoute.cost += ijEdge.cost
151     iRoute.demand += jNode.demand
152     jNode.inRoute = iRoute
153     # add jRoute to new iRoute
154     for edge in jRoute.edges:
155         iRoute.edges.append(edge)
156         iRoute.cost += edge.cost
157         iRoute.demand += edge.end.demand
158         edge.end.inRoute = iRoute
159     # delete jRoute from emerging sol
160     sol.cost -= ijEdge.savings
161     sol.routes.remove(jRoute)
162     plot_map(nodes, sol=sol, instanceName=instanceName)
163
```

cws_heuristic.py Printing the CWS Solution



```
163 """ PRINT RESULTS """
164 print(f'Instance: {instanceName}, Vehicle Capacity {vehCap}')
165 print(f'Cost of C&W savings sol = {sol.cost:.2f}')
166 for route in sol.routes:
167     s = str(0)
168     for edge in route.edges:
169         s = s + '-' + str(edge.end.ID)
170     print('Route: ' + s + ' || cost = ' + "{:.{}}f".format(route.cost, 2))
171
172 """ PLOT MAP OF NODES WITH ROUTES """
173 plot_map(nodes, sol=sol, instanceName=instanceName)
174
175
```

```
In [4]: runfile('/home/aaajp/cws_heuristic.py', wdir='/home/aaajp')
Reloaded modules: vrp_objects, vrp_visualization
Instance: A-n80-k10, Vehicle Capacity 100
Cost of C&W savings sol = 1860.94
Route: 0-1-63-10-7-21-0 || cost = 130.70
Route: 0-64-17-31-27-59-5-23-62-0 || cost = 188.89
Route: 0-24-6-30-78-8-37-2-34-71-0 || cost = 220.95
Route: 0-11-52-28-79-18-48-14-0 || cost = 193.22
Route: 0-13-74-39-60-29-44-12-0 || cost = 143.38
Route: 0-46-20-75-25-41-15-55-9-54-72-0 || cost = 248.96
Route: 0-33-47-56-69-65-35-26-19-57-61-16-43-68-0 || cost = 307.36
Route: 0-36-38-66-67-53-3-77-51-0 || cost = 145.84
Route: 0-40-42-73-49-0 || cost = 87.05
Route: 0-70-76-50-45-22-4-32-58-0 || cost = 194.59
```

vrp_visualization.py Plotting Sol (1/3)



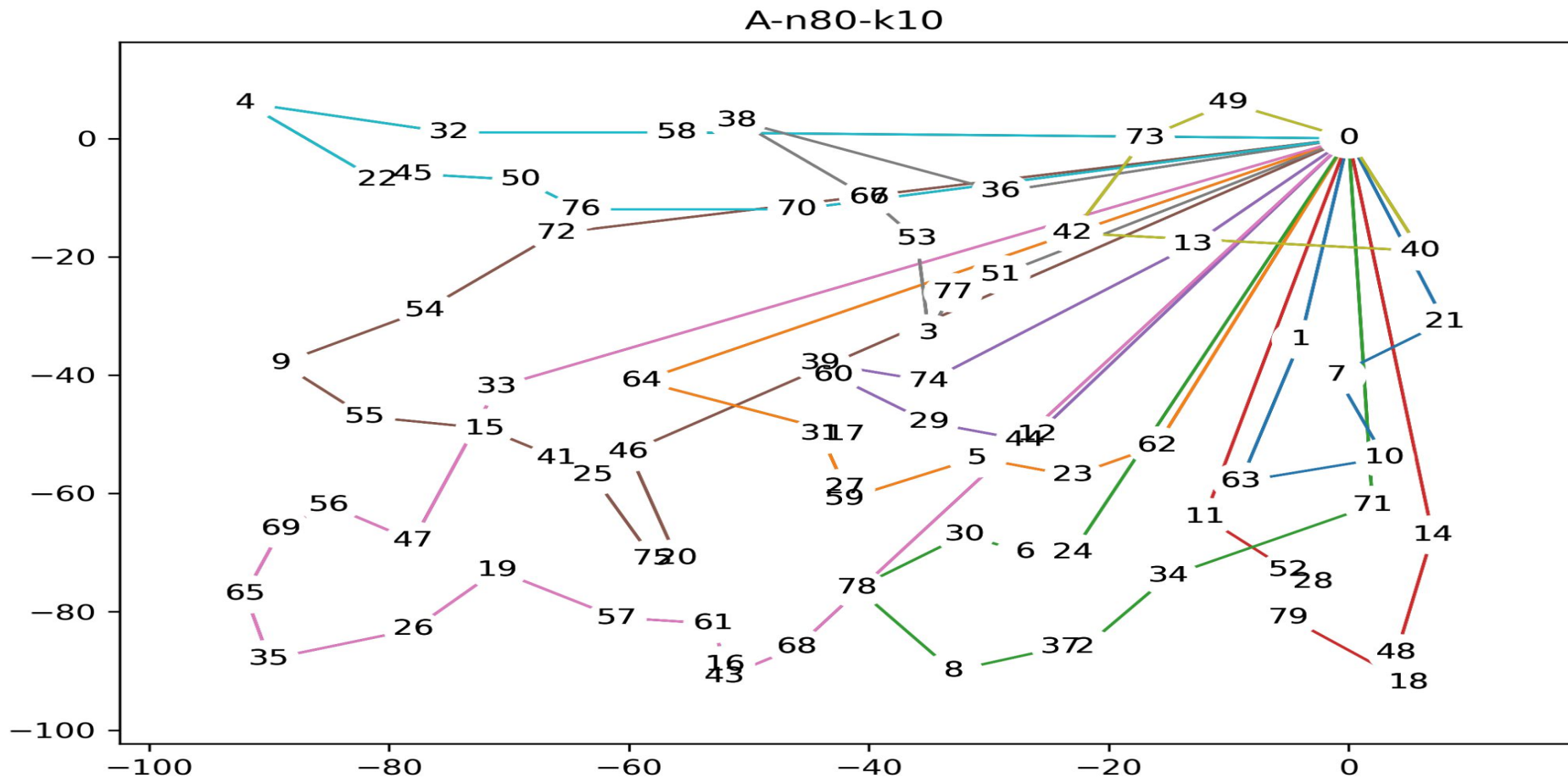
```
1 import matplotlib.pyplot as plt
2 import networkx as nx
3 import matplotlib.colors as mcolors
4
5 def plot_map(nodes, sol=None, instanceName=None, show_routes=True):
6     colors = list(mcolors.TABLEAU_COLORS)
7     figure = plt.figure(figsize=(8, 6), dpi=300)
8     G = nx.Graph()
9
10    for node in nodes:
11        G.add_node(node.ID, coord=(node.x, node.y))
12
13    if sol and show_routes:
14        i = 0
15        for route in sol.routes:
16            for edge in route.edges:
17                G.add_edge(edge.origin.ID, edge.end.ID, color=colors[i % len(colors)])
18                G.add_node(edge.end.ID, coord=(edge.end.x, edge.end.y), color=colors[i % len(colors)])
19            i += 1
20
```



NetworkX
Network Analysis in Python

```
20
21 coord = nx.get_node_attributes(G, 'coord')
22
23 if sol and show_routes:
24     col = nx.get_node_attributes(G, 'color')
25     node_color = [col[u] for u in col]
26     edges = G.edges()
27     edgecolor = [G[u][v]['color'] for u, v in edges]
28     nx.draw_networkx(G, coord, node_color=node_color, node_size=10, edge_color='black',
29                     font_color='black', font_size=10, linewidths=0.5)
30     nx.draw_networkx(G, coord, node_color='white', edge_color=edgecolor, font_color='black', font_size=10)
31 else:
32     nx.draw_networkx(G, coord, node_color='blue', node_size=10, edge_color='black',
33                     font_color='black', font_size=10, linewidths=0.5)
34
35 limits = plt.axis('on')
36 ax = figure.gca()
37 ax.tick_params(left=True, bottom=True, labelleft=True, labelbottom=True)
38
39 if instanceName:
40     plt.title(instanceName)
41
42 plt.show()
```





Homework/Lab Activities

1. Write a 3-page paper in which you describe the savings heuristics and implement it for solving the VRP, testing it in different instances and comparing your results with the ones provided in the JORS table. Include your code as appendix.

